

Python Errors, Exceptions & Debugging Master Reference

Comprehensive Notes: Anatomy of Failures, Exception Handling, Custom Errors & Diagnostic Patterns

Document Type

Lecture Reference Guide

Scope

Core Python to Advanced OOP

Target Audience

Developers & Students

Fundamental Concept: An error is an event that occurs during program execution that disrupts the normal flow of instructions. Python divides errors into three operational phases: parsing phase (Syntax Errors), execution phase (Runtime Exceptions), and logic verification phase (Logical Errors).

1. SYNTAX ERRORS (COMPILE-TIME ERRORS)

Phase 1: Parsing

Syntax errors occur when the Python parser reads code that violates the language's formal grammar rules. They are detected **before any code execution begins**. If a script contains even one syntax error, no part of the code runs.

Primary Causes & Examples:

- Unmatched Delimiters:** Missing closing brackets, parentheses, or quotes.
- Invalid Syntax Tokens:** Forgetting colons (:) after function/loop declarations or using reserved keywords incorrectly.
- Indentation Structure Failure:** Mixing spaces and tabs or misaligning code blocks.

```
# Case 1: Missing colon (SyntaxError)
if user_status == "active" # SyntaxError: expected ':'
    print("Access granted")

# Case 2: Unmatched Parentheses (SyntaxError)
total = (10 + 20 * 5      # SyntaxError: '(' was never closed

# Case 3: Block Misalignment (IndentationError)
def calculate_total(price, tax):
print("Calculating...") # IndentationError: expected an indented block
```

2. RUNTIME ERRORS (EXCEPTIONS)

Phase 2: Execution

Runtime errors occur **while the program is executing** after passing the initial syntax check. An unexpected condition causes Python to create and raise an **Exception** object. Unhandled exceptions trigger a traceback and terminate the script immediately.

Exception	Cause / Underlying Trigger	Code Example
<code>ZeroDivisionError</code>	Dividing or taking modulo of a number by zero.	<code>rate = count / 0</code>
<code>TypeError</code>	Applying an operator/function to unsupported data types.	<code>"Age: " + 25</code>
<code>ValueError</code>	Right data type received, but the value is invalid.	<code>num = int("abc")</code>
<code>NameError</code>	Attempting to access an unassigned local/global variable.	<code>print(total_score)</code>
<code>IndexError</code>	Accessing a sequence (list/tuple) index that is out of bounds.	<code>arr = [1]; arr[5]</code>

Additional Core Runtime Exceptions (Detailed Breakdown)

Exception	Cause / Underlying Trigger	Code Example
<code>KeyError</code>	Querying a non-existent key in a dictionary object.	<code>user["id_number"]</code>
<code>AttributeError</code>	Calling a method/attribute not defined for that object type.	<code>num = 10; num.append(2)</code>
<code>FileNotFoundError</code>	Opening a file system path that does not exist.	<code>open("missing.csv")</code>
<code>ImportError</code>	Failing to resolve or import a module or sub-item.	<code>from math import foo</code>
<code>RecursionError</code>	Exceeding the interpreter's maximum call stack depth.	<code>def f(): f()</code>
<code>UnboundLocalError</code>	Referencing a local variable before assigning a value to it.	<code>x += 1 # inside function</code>

3. LOGICAL ERRORS (ALGORITHMIC FLAWS)

Phase 3: Logic

Logical errors occur when code runs without crashing (no syntax or runtime errors occur) but produces **incorrect output** because the underlying logic or algorithm is flawed. Python cannot automatically detect logical errors.

```
# Flawed Logic: Order of operations error
def calculate_average(val1, val2):
    return val1 + val2 / 2 # Evaluates val2/2 first! Returns 20 instead of 15 when given (10, 20)

# Corrected Logic: Parentheses enforce intended operational priority
def calculate_average_fixed(val1, val2):
    return (val1 + val2) / 2

# Off-by-One Loop Error
items = ["a", "b", "c"]
for i in range(len(items) - 1): # Skips "c" because range end is non-inclusive!
    print(items[i])
```

4. EXCEPTION HANDLING CONTROL ARCHITECTURE

Control Flow

Python provides the `try-except-else-finally` structure to handle runtime errors gracefully without terminating program execution abruptly.

```
try:
    # Dangerous code block that might raise an exception
    file = open("user_data.txt", "r")
    val = int(file.readline().strip())
    result = 100 / val

except FileNotFoundError:
    print("Error: Targeted data file missing from disk.")

except (ValueError, ZeroDivisionError) as err:
    # Catching multiple exception types in a tuple
    print(f>Data processing failure: {err}<\/div>
<div data-bbox="95 685 607 720" data-label="Text">
<pre>
else:
    # Executes ONLY if try block runs without raising ANY exception
    print(f>Processing successful! Computed output: {result}<\/pre>
<div data-bbox="95 730 704 766" data-label="Text">
<pre>
finally:
    # ALWAYS executes (Guaranteed cleanup step regardless of success or failure)
    print(>Closing resources...<\/pre>
</div>
</div>
<div data-bbox="72 947 372 962" data-label="Page-Footer">
<div data-bbox="852 947 936 962" data-label="Page-Footer">
</div>
</div>
```

5. CUSTOM EXCEPTIONS & EXPLICIT RAISING

Advanced OOP

Developers can explicitly throw exceptions using the `raise` keyword, or construct custom domain exceptions by subclassing Python's base `Exception` class.

```
# Step 1: Define custom exception inheriting from standard Exception
class InvalidAgeError(Exception):
    """Raised when a user inputs an age outside valid biological bounds."""
    def __init__(self, age, message="Age must be between 0 and 120"):
        self.age = age
        self.message = message
        super().__init__(f"{message}. Received: {age}")

# Step 2: Raise exception in business logic
def register_voter(name, age):
    if age < 0 or age > 120:
        raise InvalidAgeError(age)
    if age < 18:
        raise ValueError(f"Candidate {name} is under voting age limit.")
    return f"Voter {name} successfully registered."
```

6. PYTHON EXCEPTION HIERARCHY

Object Tree

All standard exceptions form an object-oriented class inheritance tree rooted at `BaseException`. Understanding this hierarchy helps prevent accidental over-broad exception matching.

```
BaseException
├── SystemExit           # Raised by sys.exit()
├── KeyboardInterrupt   # Raised when user hits Ctrl+C
└── Exception           # Parent class for all standard application runtime errors
    ├── ArithmeticError
    │   ├── ZeroDivisionError
    │   └── OverflowError
    ├── LookupError
    │   ├── IndexError   # List/Tuple sequence index missing
    │   └── KeyError     # Dict key lookup failure
    ├── TypeError
    ├── ValueError
    ├── NameError
    │   └── UnboundLocalError
    ├── AttributeError
    └── OSError
        └── FileNotFoundError # Operating system level file failure
```

7. CONTEXT MANAGERS & AUTOMATIC CLEANUP

Resource Safety

Rather than relying on `finally:` blocks to close files or release database connections, use Python's `with` statement (Context Manager protocol), which guarantees cleanup even when exceptions occur.

```
# Safe file handling using context manager protocol
try:
    with open("data.txt", "r") as file:
        content = file.read()
        # File automatically closed when block exits or if an error occurs
except FileNotFoundError:
    print("File missing.")
```

1. Avoid "Bare" Except Clauses

Catching exceptions without specifying an exception class catches unexpected system calls like `KeyboardInterrupt` and `SystemExit`, making program termination difficult.

```
# BAD PRACTICE: Bare except masks all errors unconditionally
try:
    process_data()
except:
    print("An error happened!") # You don't know what failed!

# GOOD PRACTICE: Target specific exceptions explicitly
try:
    process_data()
except (ValueError, KeyError) as err:
    logger.error(f"Data processing failed: {err}")
```

2. How to Read Tracebacks Efficiently

Python prints tracebacks in **chronological order** (oldest call first, most recent call last). Read from the **bottom line up** to quickly identify the raised exception type, exact line number, and offending expression.

```
Traceback (most recent call last):
  File "app.py", line 14, in <module>
    calculate_metrics(data)
  File "app.py", line 8, in calculate_metrics
    return total / count
ZeroDivisionError: division by zero <-- START HERE (Bottom Line)
```

3. Interactive Debugging with `breakpoint()`

Insert `breakpoint()` into your code to pause execution and enter the interactive Python Debugger (`pdb`) terminal to inspect variable state live.

```
def compute_discount(price, rate):
    breakpoint() # Execution stops here; opens pdb shell
    discounted_price = price * (1 - rate)
    return discounted_price
```

Summary Checklist for Students:

- Fix `SyntaxError` issues first before searching for execution bugs.
- Keep code inside `try` blocks minimal to avoid accidentally masking unrelated bugs.
- Use standard exceptions (`ValueError` , `TypeError`) where appropriate before creating custom exception classes.